Exercise Sheet 3: Side Channels

Information Security Lab

October 30, 2026

Exercise 1: The Sound of Insecurity

Setting

You are trying to open a safe protected by a combination lock with four dials. Each dial has digits from 0 to 9, and exactly one combination opens the safe.
You do not know the combination, but the lock imposes no limit on the number of attempts. You can only test complete combinations. For each attempt, you learn whether the safe opens.

Task 1. What is the maximum number of complete combinations you need to test to determine the correct combination?

Setting

You notice that turning a dial produces a distinctive click when it reaches a certain digit. For example, the first dial clicks at 8 and the second at 5. You turn each dial until you hear a click, and the safe opens.

Task 2. What information does the clicking sound reveal? How could you use it to open another safe with the same lock model?

Task 3. What is the maximum number of individual dial positions you need to test to determine the correct combination?

Task 4. How could the manufacturer prevent or mitigate this attack?

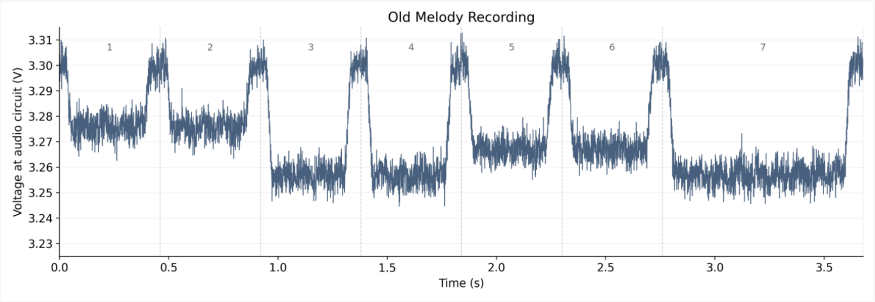
Task 5. Suppose the manufacturer introduces random clicking sounds while the dials are turning. Does this prevent the attack?

Task 6. How could a similar attack be used to extract information from a Trusted Execution Environment (TEE)?

Exercise 2: The Power of Secrets

Setting

You find an old toy instrument that can record and replay melodies. Unfortunately, its speaker is broken. You connect an oscilloscope to the output of the instrument's audio circuit and record the voltage while replaying a previously recorded melody.
You obtain the following trace:



Task 1. What information could you infer from the voltage trace? How could you test your hypothesis?

Setting

You measure the average output voltage while playing each note individually and obtain the following values:

Note	Average Voltage (V)
C	3.275
D	3.240
E	3.295
F	3.300
G	3.255
A	3.265
B	3.235

Assume that each note produces a distinguishable and stable average voltage.

Task 2. Can you recover the recorded melody from the voltage trace?

Task 3. How could a similar attack be used to extract information from a TEE?

Exercise 3: The Prime Suspect

Setting

You control the OS of a system running an Intel SGX enclave. The enclave processes a secret number n that you cannot read directly.

The enclave first performs preliminary computations taking 100 ms. It then checks whether n is prime by testing divisors in ascending order, starting at 2. The enclave stops at the first divisor of n, or after checking all integers up to $\lfloor\sqrt{n}\rfloor$. Each divisibility test takes approximately 2 ms.
Although you cannot inspect the enclave's memory, you can measure its total execution time. Assume that scheduling overhead, interrupts, and other sources of timing variation are negligible.
You observe that the enclave takes 108 ms to process the secret number.

Task 1. Assuming $n \in \{21, 24, 29, 33\}$, can you determine the secret number from the observed execution time?

Task 2. Suppose the enclave instead takes 104 ms. Can you determine the secret number?

Task 3. Does SGX prevent this timing side-channel attack? Explain your answer.

Exercise 4: It's Not My Fault!

Setting

You are analyzing an application running inside an Intel SGX enclave.

```
void common_work() {
    // Perform some computation
}

void option_A() {
    common_work();
}

void option_B() {
    common_work();
}

void option_C() {
    common_work();
}

void process_secret(int secret) {
    if (secret == 0)
        option_A();
    else if (secret == 1)
        option_B();
    else
        option_C();
}
```

The enclave processes a `secret` value that determines which of three functions is executed. As the attacker, you control the OS. Although you cannot directly read enclave memory, you can manipulate page permissions to induce and observe page faults.
Assume the following:

- Page size is 4 KiB.
- The enclave is loaded with a page-aligned load bias of `0x70000000`.
- Each function fits entirely within a single page.
- All function calls and returns occur as shown (no inlining, tail-call optimization, or other changes to the illustrated control flow).
- Assume an idealized observation model in which the OS induces an instruction page fault whenever execution enters a different code page, including when returning to a previously visited page.
- Consider only instruction page faults caused by the functions shown.
- Trace collection begins with the first instruction of `process_secret()` and ends after its final return.

Task 1. You mark an enclave code page as not present. When the enclave attempts to execute

an instruction on that page, a page fault occurs. You handle the fault and restore the page-table entry.

Will every subsequent instruction fetch from that page cause another page fault?

Task 2. You inspect the enclave binary and find that the functions are located at the following offsets relative to the enclave's load address:

Function	Offset
`process_secret()`	0x1100
`option_A()`	0x2100
`option_B()`	0x2200
`option_C()`	0x2300
`common_work()`	0x3100

Construct the instruction page-fault trace for each possible value of `secret`.

Task 3. Can you distinguish the value of `secret` from the instruction page-fault trace?

Setting

The developer modifies the enclave so that `option_C()` calls `special_work()` instead of `common_work()`.

```
void common_work() {
    // Perform some computation
}

void special_work() {
    // Perform some other computation
}

void option_A() {
    common_work();
}

void option_B() {
    common_work();
}

void option_C() {
    special_work();
}

void process_secret(int secret) {
    if (secret == 0)
        option_A();
    else if (secret == 1)
        option_B();
    else
        option_C();
}
```

Task 4. You inspect the modified enclave binary and find that the functions are located at the following offsets relative to the enclave's load address:

Function	Offset
`process_secret()`	`0x1100`
`option_A()`	`0x2100`
`option_B()`	`0x2200`
`option_C()`	`0x2300`
`common_work()`	`0x3100`
`special_work()`	`0x4100`

Construct the instruction page-fault trace for each possible value of `secret`.

Task 5. Can you distinguish the value of `secret` from the instruction page-fault trace?

Setting

The developer proposes enabling address space layout randomization (ASLR). Consider a hypothetical recompilation in which `option_A()`, `option_B()`, and `option_C()` occupy three distinct code pages, at offsets `0x2000`, `0x3000`, and `0x4000`, respectively. Instead of using a fixed load bias, the enclave is loaded at a randomly chosen page-aligned base address on each execution. Assume you do not initially know the load bias but can observe the instruction page-fault trace.

Task 6. Does randomizing the load bias prevent you from distinguishing the secret values?

Task 7. Consider the original implementation of `process_secret()`, where all three options call `common_work()`. Modify the implementation so that:

- Its behavior remains unchanged.
- The instruction page-fault trace is independent of `secret`.
- Unnecessary computation is minimized.

Provide a concrete implementation and explain why it prevents the attack.

Task 8. So far, we have assumed that the observed page-fault trace contains only accesses to the relevant enclave code pages. In practice, other code may execute and additional page faults may occur during trace collection.

How might this affect the observed trace, and what challenge does it introduce for the attacker?

Setting

To mitigate the instruction page-fault side channel, the developer changes the implementation as follows:

```
uint8_t values[8192];

uint8_t process_secret(uint16_t secret) {
    return values[secret];
}
```

Assume the following:

- The array starts at the page-aligned address `0x70005000`.
- Each array element occupies one byte.
- The secret is an integer between 0 and 8191.
- The OS can induce and observe data page faults.
- Ignore all other memory accesses.

Task 9. Does the modified implementation prevent the attacker from learning information about `secret` through page faults? If not, explain what information the attacker can learn and whether the exact secret value can be determined.